Principles of Programming Languages, 2026.01.27

Important notes
- Total available time: 2h (*multichance* students do not need to solve Exercise 1).
- You may use any written material you need, and write in English or in Italian.
- You cannot use electronic devices during the exam: every phone must be turned off and kept on your table.
- You cannot use library functions not covered in class in your code.
- Each exercise is valued 10 points, +1 bonus.

Exercise 1, Scheme

Define a for-each construct for lists, with the following syntax:

(For-each <variable> **in** <list> **do** <body>**)**

The construct may be interrupted with a **break** operation; e.g. **(break #t)** will return true.

Exercise 2, Haskell

1) Consider a classical unranked tree datatype, called **Utree**, i.e. where each node can have an unlimited number of children. Define **Utree**.

2) Consider the monadic map defined in class for binary trees, presented next. Define a monadic map for Utree.

```
mapTreeM :: Monad m => (t -> m a) -> Tree t -> m (Tree a)
mapTreeM _ Empty = do return Empty
mapTreeM f (Leaf a) = do b <- f a
                         return (Leaf b)
mapTreeM f (Branch lhs rhs) = do lhs' <- mapTreeM f lhs
                                 rhs' <- mapTreeM f rhs
                                 return (Branch lhs' rhs')
```

3) Consider the State monad seen in class; the following code is used to add a distinct value to all the leaves:

```
numberTree tree = runStateM (mapTreeM number tree) 1
    where number v = do cur <- getState
                        putState (cur+1)
                        return (v,cur)
```

Define a variant of numberTree for Utree.

Exercise 3, Erlang

Create a program that takes a single input value X and a list of different "strategy" functions [s_1, s_2, ..., s_n].

Each strategy tries to find a solution for X (e.g., different algorithms to find a prime factor, or different search heuristics). The program takes also a *timeout* value in ms.

The program should:

- Run all strategies in parallel.
- Return the result of the first strategy that finishes successfully.
- Terminate (kill) all the other processes as soon as the first result is found to save resources.
- Terminate all the process if the timeout expires, returning the atom **timeout**.

To kill a process, use the function **exit(P, kill)**, where P is the *Pid* of the process to be killed.

Solutions

Ex 1

```
(define *exit-store* '())
(define (break v)
  ((car *exit-store*) v))

(define-syntax For-each
  (syntax-rules (in do)
    ((_ var in L
        do body ...)
     (let ((out (call/cc
                  (lambda (k)
                    (set! *exit-store*
                      (cons k *exit-store*))
                    (for-each (lambda (var)
                                body ...)
                              L)))))
       (set! *exit-store* (cdr *exit-store*))
       out))))
```

Ex 2

```
1)
data Utree a = Val a | Utree [Utree a] deriving (Show, Eq)

2)
mapUtreeM :: Monad m => (a -> m b) -> Utree a -> m (Utree b)
mapUtreeM f (Val a) = do b <- f a
                         return (Val b)
mapUtreeM f (Utree ts) = do
  ts' <- mp ts
  return (Utree ts')
  where
    mp []     = return []
    mp (t:ts) = do
      t'  <- mapUtreeM f t
      ts' <- mp ts
      return (t' : ts')

3)
numberUtree tree = runStateM (mapUtreeM number tree) 1
    where number v = do cur <- getState
                        putState (cur+1)
                        return (v,cur)
```

Ex 3

```
find_fastest(Input, Strategies, Timeout) ->
    Parent = self(),
    Pids = [spawn(fun() -> Parent ! S(Input) end) || S <- Strategies],
    receive
        Result ->
            cleanup(Pids),
            Result
    after
        Timeout ->
             cleanup(Pids),
             timeout
    end.

cleanup(Pids) -> [exit(Pid, kill) || Pid <- Pids].
```